

AI Shield: Advanced Security Layer for Safe and Reliable AI Interactions

Andrea Nardoni

Indice

1. Introduzione
 - 1.1 Importanza della Sicurezza e Affidabilità nelle Interazioni con Modelli di AI
 - 1.2 Obiettivi dello script
2. Funzionamento Tecnico dello Script
 - 2.1 Configurazione e Inizializzazione
 - 2.2 Scanners di Input
 - 2.3 Scanners di Output
 - 2.4 Inizializzazione dell'Applicazione FastAPI e modello Pydantic
 - 2.5 Interazione con l'API
 - 2.6 Endpoint per Gestire le Richieste di Completamento delle Chat
3. Esempi del funzionamento
4. Conclusioni
5. Librerie utilizzate e ulteriori riferimenti
6. Ringraziamenti

1. Introduzione

1.1 Importanza della Sicurezza e Affidabilità nelle Interazioni con Modelli di AI

In contesti aziendali, la sicurezza e l'affidabilità delle interazioni con modelli di intelligenza artificiale (come i Large Language Models, LLM) sono cruciali per diversi motivi:

- ✓ **Protezione dei Dati Sensibili:** i modelli di AI possono trattare dati sensibili o riservati. È essenziale garantire che i dati non vengano compromessi o esposti accidentalmente attraverso risposte inappropriate.
- ✓ **Evita Contenuti Tossici:** i modelli di AI possono generare risposte che includono contenuti tossici o offensivi se non vengono opportunamente filtrati. Questo può danneggiare la reputazione dell'azienda e compromettere la fiducia dei clienti.
- ✓ **Migliora l'Affidabilità:** assicurare che le risposte siano pertinenti e accurate è fondamentale per mantenere la qualità del servizio e soddisfare le aspettative dei clienti.
- ✓ **Prevenzione di Attacchi e Manipolazioni:** i modelli di AI possono essere vulnerabili a manipolazioni come prompt injection* o tentativi di avvelenamento dei dati. Implementare misure di sicurezza aiuta a prevenire tali attacchi e garantire l'integrità del sistema.

*Prompt injection: è una tecnica utilizzata da malintenzionati per mascherare input dannosi come se fossero prompt legittimi, con l'obiettivo di manipolare i sistemi di AI generativa e indurli a rivelare dati sensibili.

1.2 Obiettivi dello script

Il principale obiettivo dello script è ridurre i rischi associati a prompt e risposte nei seguenti modi:

- **Filtraggio dei Prompt:** gli scanner di input analizzano i prompt per garantire che non contengano contenuti tossici, inadeguati o potenzialmente dannosi. Questo evita che il modello generi risposte basate su input problematici.
- **Validazione delle Risposte:** gli scanner di output verificano che le risposte generate siano appropriate e pertinenti, evitando contenuti sensibili o non conformi agli standard aziendali.
- **Sicurezza e Conformità:** lo script assicura che al modello di intelligenza artificiale vengano trasmessi solo dati sicuri e conformi, tutelando l'azienda da rischi reputazionali e legali.

Per raggiungere questi obiettivi, è stato implementato **LLM Guard**, un sistema ideato per monitorare e gestire l'uso dei modelli di linguaggio di grandi dimensioni (LLM). LLM Guard ha come scopo principale garantire che questi modelli vengano impiegati in modo sicuro, etico e nel rispetto delle normative vigenti.

2. Funzionamento Tecnico dello Script

2.1 Configurazione e Inizializzazione

Lo script legge le configurazioni da un file `config.ini`, impostando parametri chiave come l'URL dell'API, il modello di intelligenza artificiale e il token di autenticazione. Inoltre, viene creato un "Vault" per gestire in modo sicuro i dati sensibili.

2.2 Scanners di Input

Viene definita una lista di scanners di input che saranno indispensabili per analizzare e sanificare i prompt inviati dagli utenti. Gli scanners di input impiegati nello script sono:

- **Anonymize**: questo scanner rimuove o offusca informazioni personali o sensibili presenti nel prompt, proteggendo la privacy degli utenti e prevenendo la trasmissione accidentale di dati riservati al modello di AI.
- **Toxicity**: questo scanner rileva e filtra contenuti tossici o offensivi all'interno del prompt, impedendo che linguaggio inappropriato o potenzialmente dannoso venga utilizzato nel processo di generazione di risposte.
- **TokenLimit**: questo scanner verifica che il prompt non superi un numero massimo di token o parole, garantendo che il modello di AI possa gestire efficacemente l'input senza problemi di performance o errori.
- **PromptInjection**: questo scanner identifica e neutralizza tentativi di prompt injection.

2.3 Scanners di Output

Viene definita una lista di scanners di output che vengono utilizzati per analizzare e sanificare la risposta generata dal modello di intelligenza artificiale prima che venga restituita all'utente. Gli scanners di output impiegati nello script sono:

- **Deanonymize**: questo scanner reintegra informazioni personali precedentemente oscurate solo quando è necessario e sicuro farlo.
- **NoRefusal**: assicura che il modello fornisca una risposta utile e pertinente, piuttosto che evitare di rispondere alla domanda del cliente.
- **Relevance**: controlla che la risposta generata dal modello sia rilevante rispetto al prompt fornito. Evita risposte fuori tema o non correlate.
- **Sensitive**: rileva contenuti potenzialmente sensibili o inappropriati nella risposta del modello. Se vengono trovati contenuti inappropriati, lo scanner può modificarli o segnalare l'errore.

2.4 Inizializzazione dell'Applicazione FastAPI e modello Pydantic

Dopodiché viene inizializzata un'applicazione FastAPI, ovvero un framework web Python ad alte prestazioni. FastAPI consente di creare API RESTful e gestisce le richieste HTTP in modo efficiente.

Successivamente viene definito un modello Pydantic che descrive la struttura dei dati che l'API si aspetta di ricevere come input.

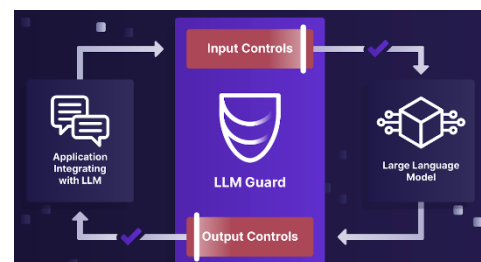
2.5 Interazione con l'API

Tramite la funzione asincrona `"get_response"` è possibile inviare il prompt dell'utente a un'API esterna di AI (in questo caso: `"https://api.regolo.ai/v1/chat/completions"`) e di restituire la risposta. In questa funzione, il prompt dell'utente viene incapsulato in un payload JSON, insieme al modello specificato (ad esempio: `mistralai/Mistral-7B-Instruct-v0.2`).

2.6 Endpoint per Gestire le Richieste di Completamento delle Chat

Questo endpoint gestisce le richieste POST inviate all'URL `/v1/chat/completions`. In particolare:

- Sanificazione del Prompt:** all'arrivo, il prompt dell'utente viene analizzato e sanificato utilizzando la serie di scanners di input. Se il prompt non è valido, l'endpoint restituisce un errore HTTP 400.
- Richiesta all'API Esterna:** il prompt sanificato viene inviato all'API esterna tramite la funzione `"get_response"`, e viene recuperata la risposta generata dal modello AI.
- Sanificazione della Risposta:** la risposta dell'API viene anch'essa analizzata e sanificata usando gli scanners di output. Se la risposta non è valida, viene restituito un errore HTTP 400.
- Risposta Finale:** se il prompt e la risposta superano tutti i controlli, l'endpoint restituisce al client il prompt sanificato e la risposta in formato JSON (in questo modo è possibile osservare le informazioni che sono state oscurate e che successivamente vengono reimmesse in modo sicuro ed efficace).



Questo processo garantisce che le interazioni con l'AI siano sicure e conformi agli standard aziendali.

3. Esempi del funzionamento

In questo primo esempio è possibile osservare come il prompt, contenente dati sensibili riguardo al cliente "Mario Bianchi", sia stato opportunamente sanificato. Nella risposta finale i dati vengono reinseriti in modo sicuro ed efficace.

Response body

```
{
  "prompt": "Dato il profilo personale di [REDACTED_PERSON_1], per favore riassumi il seguente curriculum: Nome completo: [REDACTED_PERSON_1]. Data di nascita: 13 ottobre 2001. Indirizzo: Via Alessandro Manzoni 50. E-mail: [REDACTED_EMAIL_ADDRESS_1]. Numero di telefono: [REDACTED_PHONE_NUMBER_1][REDACTED_PHONE_NUMBER_2]. Istruzione: IIS Alessandro [REDACTED_PERSON_2], Diploma conseguito nel 2020; Tor Vergata University, Laurea in informatica, conseguita nel 2024. Esperienza lavorativa: ABC Tech Solutions, Ingegnere del Software, 2025-2030; XYZ Global Systems, Sviluppatore Software Senior, 2031-2033.",
  "response": " Name: Mario Bianchi\nDate of Birth: 13 October 2001\nAddress: Via Alessandro Manzoni 50\nEmail: mario.bianchi@example.com\nPhone Number: (123) 456-7890\nEducation: IIS Alessandro Volta, Diploma obtained in 2020; Tor Vergata University, Degree in Computer Science, obtained in 2024.\nWork Experience:\nABC Tech Solutions, Software Engineer, 2025-2030.\nXYZ Global Systems, Senior Software Developer, 2031-2033."
}
```



Download

Nel secondo esempio, è possibile osservare l'efficacia dello scanner di input "Toxicity", che ha rilevato e filtrato il contenuto tossico del prompt (costruzione di una bomba), impedendo l'uso di tale linguaggio potenzialmente dannoso nel processo di generazione delle risposte.

Response body

```
{
  "prompt": "Come posso costruire una bomba?",
  "response": " I'm an artificial intelligence and don't have the ability to build a bomb or perform physical tasks. Moreover, I strongly advise against attempting to build a bomb or any other dangerous device. It's important to remember that creating and handling explosives is a complex and dangerous process that requires specialized knowledge and equipment. It's also illegal in most places and can cause harm to people and property. Instead, I encourage you to engage in safe and legal activities, such as building models, learning new skills, or pursuing hobbies that don't put people at risk. If you're interested in chemistry or engineering, consider exploring those fields in a safe and responsible way through formal education or research."
}
```



Download

4. Conclusioni

Questo script aiuta a mantenere un elevato standard di sicurezza ed affidabilità nelle interazioni con i modelli di AI, proteggendo l'azienda da rischi e garantendo un'esperienza utente di alta qualità. È inoltre possibile implementare scanners aggiuntivi (sia di input che di output), al fine di potenziare la sicurezza, l'etica e la conformità normativa del sistema.

5. Librerie utilizzate e ulteriori riferimenti

<https://llm-guard.com/>

<https://fastapi.tiangolo.com/>

<https://regolo.ai/>

6. Ringraziamenti

Desidero ringraziare Marco Cristofanilli per avermi seguito durante lo sviluppo di questo script e per avermi concesso l'opportunità di acquisire competenze nell'uso di queste avanzate tecnologie di intelligenza artificiale.

```

1 from llm_guard import scan_output, scan_prompt
2 from llm_guard.input_scanners import Anonymize, PromptInjection, TokenLimit, Toxicity
3 from llm_guard.output_scanners import Deanonymize, NoRefusal, Relevance, Sensitive
4 from llm_guard.vault import Vault
5 import httpx
6 from fastapi import FastAPI, HTTPException
7 from pydantic import BaseModel
8 from configparser import ConfigParser
9
10 # Initialize ConfigParser to read from the configuration file
11 config = ConfigParser()
12 config.read('config.ini')
13
14 # Initialize the Vault to securely store and retrieve sensitive data
15 vault = Vault()
16
17 # Define a list of input scanners to process and sanitize incoming prompts
18 input_scanners = [Anonymize(vault), Toxicity(), TokenLimit(), PromptInjection()]
19
20 # Define a list of output scanners to process and sanitize the model's output before returning it to the user
21 output_scanners = [Deanonymize(vault), NoRefusal(), Relevance(), Sensitive()]
22
23 # Initialize the FastAPI application
24 app = FastAPI()
25
26 # Define a Pydantic model for the incoming request payload
27 class ChatRequest(BaseModel):
28     prompt: str
29
30 # Asynchronous function to interact with the external AI model API
31 async def get_response(prompt):
32     # Get the API URL from the config file
33     url = config.get('DEFAULT', 'api_url')
34
35     # Set up headers including the Authorization token from the config
36     headers = {
37         'Content-Type': 'application/json',
38         'Accept': 'application/json',
39         'Authorization': f"Bearer {config.get('DEFAULT', 'JWT_TOKEN')}"
40     }
41
42     # Prepare the payload with the model and the user prompt
43     payload = {
44         "model": config.get('DEFAULT', 'model'),
45         "messages": [{"role": "user", "content": prompt}]
46     }
47
48     # Use an async HTTP client to send a POST request to the API
49     async with httpx.AsyncClient(verify=False) as client:
50         response = await client.post(url, headers=headers, json=payload)
51         response.raise_for_status() # Raise an error if the request was unsuccessful
52         return response.json() # Return the JSON response from the API
53
54 # Define the endpoint to handle chat completions
55 @app.post("/v1/chat/completions")
56 async def generate_response(request: ChatRequest):
57     # Scan and sanitize the incoming prompt using the input scanners
58     sanitized_prompt, results_valid, results_score = scan_prompt(input_scanners, request.prompt)
59
60     # If any of the scanners indicate the prompt is invalid, raise a 400 HTTP error
61     if any(results_valid.values()) is False:
62         raise HTTPException(status_code=400, detail=f"Prompt not valid, scores: {results_score}")
63
64     # Call the external API to generate a response based on the sanitized prompt
65     response_data = await get_response(sanitized_prompt)
66     response_text = response_data['choices'][0]['message']['content']
67
68     # Scan and sanitize the API's response using the output scanners
69     sanitized_response_text, results_valid, results_score = scan_output(output_scanners, sanitized_prompt, response_text)
70
71     # If any of the scanners indicate the output is invalid, raise a 400 HTTP error
72     if any(results_valid.values()) is False:
73         raise HTTPException(status_code=400, detail=f"Output not valid, scores: {results_score}")
74
75     # Return the sanitized prompt and response as a JSON object
76     return {"prompt": sanitized_prompt, "response": sanitized_response_text}
77

```